Get type name
How to get the type as a string: https://stackoverflow.com/questions/5354431/how-to-get-the-type-as-a-string-in-haskell
Use **typeOf** or **typeRep** in Data.Typeable

```
import Data.Typeable

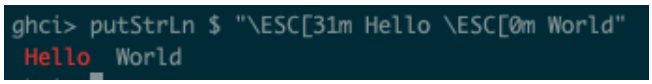
typeOf :: Typeable a => a -> TypeRep                --
value
typeRep :: Typeable a => proxy a -> TypeRep         --
phantom type

-- typeOf to get name from value
typeOf 1 :: TypeRep
show $ typeOf 1                -- "Integer"
show $ typeOf True             -- "Bool"

-- typeRep to get name from type
getName :: forall m a. (Typeable m) => m a -> String
getName _ = show $ typeRep (Proxy @m)
getName (Just 1)                  -- "Maybe"
getName (print 1)                 -- "IO"
```

Print with ANSI colors
Ref: https://stackoverflow.com/questions/21220142/change-color-of-a-string-haskell
Use "\ESC[" + ANSI code

```
-- e.g.
ghci> putStrLn $ "\ESC[31m Hello \ESC[0m World"
```



Print content of a functor e.g. list
traverse_ to apply print to each value + collapse into IO ()

```
traverse_ print [1..10]
traverse_ print (Just 1)         -- e.g. Maybe Functor

traverse_ print (InL $ Just 1)
```

[Expression] avoiding explicit if/else and case expressions

```
-- case: handling Maybe
maybe :: b -> (a -> b) -> Maybe a -> b
case x of                                    -- OK: Usual
way
  Just v -> f v
  Nothing -> x0
maybe x0 f x                                 -- OK:
Concise
-- e.g. convert Maybe -> Either
```

```
maybe (Left "Error") return (Just 5)        -- Right 5
maybe (Left "Error") return Nothing         --
Left "ERROR"


-- case: handling Either
either :: (a -> c) -> (b -> c) -> Either a b -> c
case x of                                   -- OK: Usual
way
  Left v -> f v
  Right v -> g v
either f g x                                -- OK:
concise


-- case: exit early from a domain
unless :: Applicative f => Bool -> f () -> f()
when :: Applicative f => Bool -> f () -> f()
-- e.g. exit early in a Maybe
do
 if ...condition... then Nothing else return ()
 when ...condition... Nothing
 unless not (....condition...) Nothing
```

[data] Convert between numerical types

In general, 4 different types of numbers:
- **signed, bounded integer**: e.g. Int, Int8
- **unsigned, bounded integer**: e.g. Word, Word8
- **unbounded integer**: i.e. Integer
- **floating point**: e.g. Float, Double

5 conversion methods:
- **fromIntegral**: any integer -> bounded integer
- **toInteger**: any bounded integer -> unbounded integer
- **fromInteger**: unbounded integer -> any
- **round**: floating point -> any integer, both bounded/unbounded
- **realToFrac**: any -> floating point

```
vint = 1 :: Int                 -- signed integer
vword = 1 :: Word               -- unsigned integer
vint8 = 1 :: Int8               -- 8-bit signed integer
vinteger = 1 :: Integer         -- unbounded integer
vfloat = 1.1 :: Float
vdouble = 1.1 :: Double

-- bounded integer -> real number
realToFrac vint + vdouble           -- Int -> Double
fromIntegral vint + vdouble         -- OK, also works.
Int -> Double
realToFrac vint + vfloat            -- Int -> Float
realToFrac vint8 + vdouble          -- Int8 -> Double
realToFrac vword + vdouble          -- Word -> Double

-- real number -> bounded integer
```

```
vint + round vdouble            -- Double -> Int
vint + round vfloat             -- Float -> Int
vint8 + round vdouble           -- Double -> Int8
vword + round vdouble           -- Double -> Word

-- longer -> shorter integer
fromIntegral vint + vint8       -- Int -> Int8
fromIntegral vword + vint8      -- Word -> Int8
-- shorter -> longer integer
vint + fromIntegral vint8       -- Int8 -> Int
vword + fromIntegral vint8      -- Int8 -> Word

-- signed integer -> unsigned integer
fromIntegral vint + vword       -- Int -> Word
vword + fromIntegral vint8      -- Int8 -> Word
-- unsigned integer -> signed integer
vint + fromIntegral vword       -- Word -> Int
fromIntegral vword + vint8      -- Word -> Int8

-- any other -> unbounded integer
toInteger vint + vinteger       -- Int -> Integer
toInteger vint8 + vinteger      -- Int8 -> Integer
toInteger vword + vinteger      -- Word -> Integer
round vdouble + vinteger        -- Double -> Integer

-- unbounded integer -> any other
vint + fromInteger vinteger     -- Integer -> Int
vint8 + fromInteger vinteger    -- Integer -> Int8
vword + fromInteger vinteger    -- Integer -> Word
vdouble + fromInteger vinteger  -- Integer -> Double
vdouble + fromIntegral vinteger -- Also works. Integer
-> Double

-- real -> real
vfloat + realToFrac vdouble     -- Double -> Float
realToFrac vfloat + vdouble     -- Float -> Double

-- mix
vdouble + fromIntegral vint + fromIntegral vinteger +
realToFrac vfloat     -- Float,Int,Integer -> Double
vint + fromIntegral vint8 + fromIntegral vinteger +
round vfloat            -- Float,Int8,Integer -> Int
vinteger + toInteger vint + round vfloat
                    -- Float,Int -> Integer
```

Literals (String, Num, List) for complex times

Define literal with a **function**

```
data MyData a = MyData String a

instance IsString (a -> MyData a) where
  fromString = MyData
```

Function literal

```
"abc" True :: MyData Bool
```

OK

e.g. literal MyData from String

```
{-# LANGUAGE OverloadedStrings #-}

import GHC.Exts (IsString (..))

data MyData a = MyData String a
  deriving (Show, Eq)

instance IsString (a -> MyData a) where
  fromString = MyData
```

e.g. literal Date

```
{-# LANGUAGE TypeOperators #-}

import Data.Time

instance (a ~ Int, b ~ Int) => Num (a -> b -> Day)
where        -- GOOD
 fromInteger = fromGregorian
x = 2024 1 30 :: Date

instance Num (Int -> Int -> Day) where
         -- BAD
 fromInteger = fromGregorian
x = 2024 1 30 :: Date
         -- ERROR. Cannot infer type of 1 and 30
x = 2024 (1 :: Int) (30 :: Int) :: Date
```

[list] Collapse a list

```
-- trick: collapse list
concat [[1,2,3],[4,5,6]]
join [[1,2,3],[4,5,6]]
msum [[1,2,3],[4,5,6]]
[[1,2,3],[4,5,6]] >>= id
```

[IO] Get absolute path of current file/directory
Ref: https://stackoverflow.com/questions/25737117/haskell-this-files-path
Need templates
Use **Loc** data of templates to get path of current module
Use **makeAbsolute** of System.Directory

```
{-# LANGUAGE TemplateHaskell #-}
import Data.Functor
import Language.Haskell.TH
import System.Directory

-- e.g. /a/b/c/Lib.hs
```

```
thisFile =
  $( location                                          --
Step 1: location of current module
      >>= runIO . makeAbsolute . loc_filename        --
Step 2: make absolute
      <&> LitE . StringL
   )

-- e.g. /a/b/c
import System.FilePath
thisDir :: String
thisDir = takeDirectory thisFile
```

==========

Functions

Constant function

```
-- Constant function
const 1
return 1
```

Function composition

```
-- Function composition
--
-- 4 methods
-- method 1) (.)
h . g . f            -- h(g(f(x))

-- method 2) <$>
h <$> g <$> f        -- with functor

-- method 3) >>=      -- monad
do
 y <- f
 return $ h (g y)

-- method 4)         -- arrow
import Control.Arrow
f >>> g >>> h
```

Compose a function N times

Use foldr

```
-- e.g.
foldr ($) x (replicate 3 f)         -- f(f(f(x))
foldr ($) 1 (replicate 3 (*2))      -- 2*2*2=8
```

Apply/compose function in natural order

Use &, <&> for application

Use arrow >>> for composition

```
---- Apply
--
-- solution: & operator
x               -- natural order with &
 & f            -- f(x)
 & g            -- g(f(x))
 & h            -- h(g(f(x))
h $ g $ f x     -- usual h(g(f(x))  i.e. reverse
(h . g . f) x   -- same

-- functor
mx
 <&> f
 <&> g
 <&> h
h <$> g <$> f mx      -- same
(h.g.f) <$> mx        -- same


---- Compose
--
-- solution: >>> operator
import Control.Arrow ((>>>), (<<<))
f >>> g >>> h         -- natural order h · g · f with >>>
f >>> g >>> h $ x     -- h(g(f(x))
h . g . f             -- usual h · g · f i.e. reverse
h . g . f $ x
h <<< g <<< f         -- same


-- Kliesli composition
--
-- k: a -> m b
-- h: b -> m c
-- l: c -> m d
k >=> h >=> l         -- composition
l <=< h <=< k         -- natural
```

Compose domain (i.e. contravariant) on a multivariate function i.e. u(f(x),g(x),h(x))
Use applicatives <*> to apply g1, g2

```
-- <*> to compose each term
--
u <$> f1                            -- u(f1(x))
pure u <*> f1                       -- same
u <$> f1 <*> f2                     -- u(f1(x),f2(x))
u <$> f1 <*> f2 <*> f3              --
u(f1(x),f2(x),f3(x))
u <$> f1 <*> f2 <*> f3 ... <*> fn     --
u(f1(x),...,fn(x))
u <$> f1 <*> f2 <*> f3 ... <*> fn     --
```

```
u(f1(x),...,fn(x))
do                                  -- with do
 y1 <- f1
 y2 <- f2
 ...
 return (f y1 y2 ... )

-- e.g. binary
(*) <$> (+1) <*> (+2)                       --
(x+1)*(x+2)
(*) <$> (+1) <*> ((*) <$> (+2) <*> (+3))    --
(x+1)*(x+2)*(x+3).
```

Compose range of a multivariate function i.e. g(f(u(x,y,z)))
Use . .

```
f . g . u                  -- f(g(u(x))
((f . g) .) . u            -- f(g(u(x,y))
(((f . g) .) .) . u        -- f(g(u(x,y,z)))
(((f . g) .) .) . ) . u    -- f(g(u(x,y,z,a)))
... etc
```

Chain function compositions

```
-- chain compositions
--
-- e.g.
(#*) f g = (*) <$> f <*> g             -- compose with
(*)
(#+) f g = (+) <$> f <*> g             -- compose with
(+)
(#-) f g = (-) <$> f <*> g             -- compose with
(-)
(+1)#*(+2)                             -- (x+1)*(x+2)
(+1)#*((+2)#*(+3)                      --
 (x+1)*(x+2)*(x+3)
(+1)#*((+2)#*((+3)#*(+4)))             --
 (x+1)*(x+2)*(x+3)*(x+4)
(10*)#+((+1)#*((+2)#*((+3)#*(+4))))    -- (10x)+
(x+1)*(x+2)*(x+3)*(x+4)
(10*)#+((+1)#*((+2)#*((+3)#*(+4))))$1  -- 10+2*3*4*5 =
130
```

Bidirectional parsing (conversion) with **inverse map** with bounded enums
Ref: https://kowainik.github.io/posts/haskell-mini-patterns#bidirectional-parsing
[2026/01] Haskell idioms we like: https://exploring-better-ways.bellroy.com/some-haskell-idioms-we-like.html

Problem
Converting between 2 types is usually not **bijective**. In particular, the conversion is only **well-defined** in **one direction**.
A problem with this is that **pattern matching** is **exhaustive** only in one direction. i.e. can **type check** for **missed cases** only in one direction
e.g. Bool <-> String conversion

Bool -> String **well-defined** and hence **exhaustive pattern matching** i.e can detect missed cases

String -> Bool not **well-defined** (e.g. "a" is not an Bool) and hence no exhaustivity check i.e. cannot detect missed cases.

Solution
If the pattern-matchable values are **finitely enumerable** (finite Enum), then can **inverse map** to safely reverse convert.
Simply check all enum values until a match is found.

Inverse map

```
-- s       value to invert
-- e -> s  enum map for comparision
inverseMap :: forall e s. (Eq s, Enum e, Bounded e) =>
(e -> s) -> s -> Maybe e
inverseMap enumMap input = foldr checkResult Nothing
allValues
 where
  checkResult _ x@(Just _) = x
  checkResult value Nothing = if enumMap value == input
then Just value else Nothing
  allValues = enumFromTo (minBound @e) (maxBound @e)
```

e.g. inverse map

```
data Alpha = A | B | C
  deriving (Eq, Bounded, Enum, Show)

toString :: Alpha -> String                    -- GOOD
safe Alpha -> String with exhaustive pattern matching
toString A = "a"
toString B = "b"
toString C = "c"

fromString :: String -> Maybe Alpha            -- GOOD
type safe String -> Alpha with inverse map + toString
fromString = inverseMap toString

fromString_ :: String -> Maybe Alpha           -- BAD
unsafe String -> Alpha
fromString_ "a" = Just A
fromString_ "b" = Just B                       -- OOPS!
C is missed
fromString_ _ = Nothing
```

==========
Types / typeclass

Error implicitly inferring type of a subexpression e.g. "Could not deduce ~

arising from a use of ~"
Ref: https://stackoverflow.com/questions/69965760/type-deduce-error-for-maxbound-in-a-method-with-constraints

Problem: the type of a value may not be **implicitly inferred**.
e.g.

```
getMaxEnum :: (Bounded a, Enum a) => a -> Int
getMaxEnum _ = fromEnum (maxBound)                --
ERROR

"Could not deduce (Enum a0) arising from a use of
'fromEnum' from the context: (Enum a, Bounded a) bound
by the type signature for: getMaxEnum
```

Solution: **explicitly** specify type.

```
--
-- e.g.
getMaxEnum :: (Bounded a, Enum a) => a -> Int
getMaxEnum _ = fromEnum (maxBound)                --
ERROR


-- Solution 1: use signature
getMaxEnum :: (Bounded a, Enum a) => a -> Int
getMaxEnum (_::a) = fromEnum (maxBound::a)        --
maxBound :: a

-- Solution 2: use asTypeOf
getMaxEnum :: (Bounded a, Enum a) => a -> Int
getMaxEnum x = fromEnum (maxBound `asTypeOf` x)

-- Solution 3: use type application (GHC Ext)
-- Requires GHC extensions ScopedTypeVariables,
TypeApplications
{-# LANGUAGE ScopedTypeVariables, TypeApplications #-}
getMaxEnum :: forall a. (Enum a, Bounded a) => a -> Int
getMaxEnum _ = fromEnum (maxBound::a)             -- OK
getMaxEnum _ = fromEnum (maxBound @a)             -- OK
```

Apply boilerplate typeclass instance with deriving via
Ref: https://magnus.therning.org/2023-01-15-composing-instances-using-~deriving-via~.html
Create a **container type** implementing the typeclass.
Copy implementation by deriving via the container.
Allows multiple

```
{-# LANGUAGE DerivingVia #-}

-- e.g. classes to implement
class MyClass1 a where                 -- nullary type a
  something :: a -> a
class MyClass2 m where                 -- unary type m
```

```
  something2 :: m a -> a

-- step: create model implementation of MyClass1
newtype A a = A a
instance MyClass1 (A a) where
  ...
newtype A2 a = A2 a                    -- another
implementation
instance MyClass1 (A2 a) where
  ...

-- step: create model implementation of MyClass2
newtype B m a = B (m a)
instance MyClass2 (B m) where
  ...

-- step: derive via model
data SomeType a = SomeType a
  deriving MyClass1 via (A (SomeType a))            --
copy MyClass1 via A
  deriving MyClass1 via (A2 (SomeType a))           --
OR copy MyClass1 via A2
  deriving MyClass2 via (B SomeType)                --
copy MyClass2

-- same with standalone
deriving via (A (SomeType a)) instance MyClass1
(SomeType a)
deriving via (B SomeType) instance MyClass2 SomeType
```

==========

Monad

Kliesli composition i.e. composition for >>=

Use

```
>=> :: Monad m => (a -> m b) -> (b -> m c) -> (a -> m c)
```

```
-- e.g.
myFunction :: ma -> m d
myFunction x = x >>= k >>= h >>= j        -- k::a -> m b
  h::b -> m c   j::c -> m d
myFunction = k >=> h >=> j                -- same
```

Combine multiple values into a single complex value

Use applicative **<*>** + **data constructor**

```
-- e.g. combine into tuple
(,) <$> Just 1 <*> Just 2                -- Just (1,2)
(,,) <$> Just 1 <*> Just 2 <*> Just 3    -- Just (1,2,3)

-- e.g. combine into list
(:[]) <$> Just 1
    -- Just [1]
```

```
(:) <$> Just 2 <*> ((:[]) <$> Just 1)
   -- Just [1,2]
(:) <$> Just 3 <*> ((:) <$> Just 2 <*> ((:[]) <$> Just
1))  -- Just [1,2,3]

-- e.g. combine into custom type
data A = A Int String Double
A <$> Just 1 <*> Just "Yui" <*> Just 1.1        -- A
1 "Yui" 1.1
```

For/While loop

```
-- for
forM :: (Monad m, Traversable t) => t a -> (a -> m b) ->
m (t b)
forLoop :: (Monad m, Traversable f) => m (t a) -> (a ->
m b) -> m (t b)
forLoop m f = m >>= traverse f

-- e.g.
forLoop (Just [1..10]) (Just.(*2))        -- Just
[2,4..]

-- while
whileLoop :: Monad m => m Bool -> m () -> m ()
whileLoop p m = p >>= flip when (m >> whileLoop p m)
```

[Maybe/Either] convert Maybe <-> Either

```
-- Maybe -> Either
maybeToEither :: a -> Maybe b -> Either a b
maybeToEither e = maybe (Left e) Right
eitherToMaybe :: Either a b -> Maybe b
eitherToMaybe = either (Nothing) Just
```

[Maybe/Either] exit early if error
Use when/unless

```
-- Maybe a
do
 unless (...condition1...) Nothing
 unless (...condition2...) Nothing
 when (not ...condition1...) Nothing        -- same with
when
 when (not ...condition2...) Nothing

 ...

-- Either a b
do
  unless (...condition1...) $ Left ...error...
  unless (...condition2...) $ Left ...error...
  when (not ...condition1...) $ Left ...error...
```

```
    -- same with when
  when (not ...condition2...) $ Left ...error...
```

[Maybe/Either] pass value through validation pipeline
Use when/unless

```
-- Maybe a
--
value :: Maybe a
validate1 = unless (...condition...) Nothing
validate2 = unless (...condition...) Nothing
validate1 = when (not ...condition...) Nothing
    -- same with when
validate2 = when (not ...condition...) Nothing
value >> validate1 >> validate2 >> value
    -- may fail at any point i.e. value=Nothing,
validate1=Nothing, or validate2=Nothing

-- Either a b
result :: Either a b
validate2 = unless (...condition...) $
Left ...error...
validate1 = when (not ...condition...)
$ Left ...error...   -- same with when
result >> validate1 >> validate2 >> result
    -- may fail at any point i.e. value=Left *,
validate1=Left *, or validate2=Left *
```

[State] combining multiple states (nesting states)
Ref: http://blog.sigfpe.com/2006/05/grok-haskell-monad-transformers.html

Use **nested states**.
Use lift to access inner states.
e.g.

get	state of first layer
lift $ get	state of second layer
...	

```
import Controls.Monads.Trans.Class
import Controls.Monads.Trans.State

-- e.g. state (Int, String, Bool)
--
type MyBadTestState = State (Int, String, Bool) ()
        -- Works but adhoc
type MyTestState = StateT Int (StateT String (State
Bool)) ()    -- Better

task :: MyTestState
task = do
  -- read state
  num <- get                        -- read first state
layer. i.e. Int state
  str <- lift get                   -- String state
```

```
  flag <- lift $ lift get           -- Bool state

  -- write
  put (num + 1)
  modify (+1)                      -- same with modify
  lift $ put ("Hello" ++ str)
  lift $ modify ("Hello"++)        -- same
  lift $ lift $ put (not flag)
  lift $ lift $ modify not

result = runState (runStateT (runStateT task  0) "Yui")
True     -- result :: ((((), Int), String), Bool)
((((), num), str), flag) = result
         -- num=2 str="HelloYui" flag=False
```

[State] passing IO as state
Ref: http://blog.sigfpe.com/2006/05/grok-haskell-monad-transformers.html

No need to wrap **IO** in a StateT.
Can **access IO directly** without get/put
e.g.
  lift $ putStrLn ""
  lift $ getLn

```
import Controls.Monads.Trans.Class
import Controls.Monads.Trans.State

-- e.g.
type MyIOState = StateT Int (StateT String (State IO))
()        -- No need to wrap IO
type MyIOState = StateT Int (StateT String IO) ()
         -- GOOD

iotask = do
  -- Other states
  modify (+1)                    -- Int state
  lift $ modify ("Hello"++)      -- String state

  -- IO
  -- No need to use put/get, but
  lift $ lift $ putStrLn "Press any key"        -- IO
put directly
  c <- lift $ lift $ getChar                    -- IO
get directly
  lift $ lift $ putStrLn $ "input:" ++ [c]

let
  result = runStateT (runStateT iotask 1) ""      --
result :: IO ((((), Int), String)
  in do
    ((((), num), str) <- result                   --
num=2 str=Hello
```

```
# console
```

```
Press any key
c
input:c
```